Selecting Securities (Users)

Due to the specialness of securities (commodities), both on the technical level and on the business-logic level, there are several things you must understand/keep in mind before working with them.

In a nutshell, it boils down to the following:

- *Technical level*: Securities have no technical IDs in GnuCash. Instead, they are technically selected with a (pseudo-)technical ID, constisting of a namespace and a code.
- *Business-logic level*: In the real world out there, you normally would identify a security by its security code, which hopefully you have used when entering the security in the GnuCash file:
  - If you live in the US or Canada, then you would typically use the 1-to-4-char ticker (such as "T" or "MSFT"). But strictly speaking, this is not enough. It always has to be qualified with the according exchange (such as "NYSE_AMERICAN" (formerly known as "AMEX") or "NASDAQ"). (The pros among you might use the CUSIP intead.)
  - If you live outside of the US and Canada, then you would typically use something else. In the EU, where the current maintainer lives, people typically use the ISIN (international). In Germany, the nostalgic folks prefer the WKN. Similarly, in the rest of Europe: The SEDOL (UK) or the VALOR (Switzerland), etc. etc.

  If you have a global portfolio, it makes sense to use the ISIN.[1]

Overview

Figure 1 shows the two levels of Security IDs in GnuCash:

In theory, the two layers are clearly separated, i.e. technical IDs and business-logic ID do not have anything to do with each other. However, the GnuCash developers chose to make things a little more complicated: They intentionally blurred the line between the two layers, so that technical IDs are, in fact, pseudo-technical and quasi-business.

If they had chosen to treat securties as any other entity in GnuCash, then the technical ID would look something like this: `14305dc80e034834b3f531696d81b493`. This string, obviously has no meaning, i.e. it cannot / shall not be "interpreted" or "understood".

Well, they chose another approach: In GnuCash, a (pseudo-)technical ID looks something like these:

- EURONEXT:SAP

[1] This is how thes current maintainer does it in his own portfolio, and it's been working well for decades now.

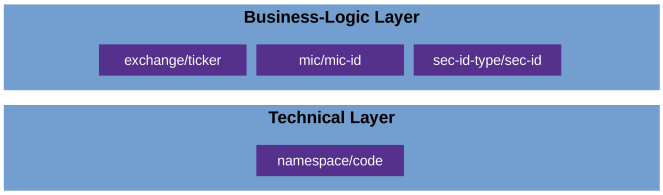

Figure 1: ID layers in GnuCash

- ISIN:DE000BASF111AP

Just by glancing at these, you can see that they are essentially business-logic IDs maskerading as technical ones; they do mean something (i.e., they have semantics) and can thus very well be interpreted and understood.

It is important to understand this (non-)difference between technical and business-logic IDs in GnuCash before reading the next section; otherwise it will probably confuse you.

You can also see that the two examples above are composed of two parts, which is no coincidence but the system that GnuCash imposes: `<namespace>:<code>`. Whereas the name space can, in theory, be freely chosen by the user, it makes sense not to do so but instead choosing from a pre-defined set that `JGnuCashLib` provides. This then leads to the concept of providing IDs "indirectly", i.e. composing them: providing the name space and the code separately.

Using the Tools

We use the test data file provided with module "API Extensions".[2] For test and illustration purposes, we have put securities into this file using various different

[2]The test data files of the other modules will almost certainly work as well – they all are very similar.

systems (i.e., name spaces) – something you normally would not do in real life.

We have provided a wrapper script for the according tool: ::TODO

Then, you have several options:

- Specifying the security by its (pseudo-)technical ID (directly), using the syntax `<namespace>:<code>`.

  This approach always works, but does not leverage the pre-defined name spaces. In case of an error, you won't get precise logs and could possibly spend a lot of time finding it.

  Example:

  ```
  $ gcsh_get_sec_info.sh \
      -f test.gnucash \
      -ssm ID \
      -sssm DIRECT \
      -sec EURONEXT:SAP
  ```

  or

  ```
  $ gcsh_get_sec_info.sh \
      -f test.gnucash \
      -ssm ID \
      -sssm DIRECT \
      -sec ISIN:DE000BASF111AP
  ```

- Specifying the security by its (pseudo-)technical ID (indirectly), using the various command line options.

  This approach leverages the pre-defined name spaces, and will thus lead to far more precise error logs in case of an error.

  Example:

  ```
  $ gcsh_get_sec_info.sh \
      -f test.gnucash \
      -ssm ID \
      -sssm INDIRECT_EXCHANGE_TICKER \
      -exch EURONEXT \
      -tkr SAP
  ```

  or

  ```
  $ gcsh_get_sec_info.sh \
      -f test.gnucash \
      -ssm ID \
      -sssm INDIRECT_SEC_ID_TYPE \
      -secid-type ISIN \
      -is DE000BASF111AP
  ```

Notice that the second example uses the ISIN, which in this case is used to specify the securitie's (pseudo-)technical ID (because the parameter `-ssm` is set to "INDIRECT_SEC_ID_TYPE").

Also notice the parameters `-exch EURONEXT` and `-secid-type ISIN` in the examples. These are, as already stated, *pre-defined* values of `JGnuCashLib`, and strictly speaking, they have nothing to do with what you actually have in your GnuCash file (obviously, it is strongly recommended that you use these pre-defined values when editing securities in GnuCash).[3] But if, for example, in one of your file's securities you wrote, say, "EURONXT" instead of "EURONEXT" by accident, then you would not be able to retrieve that one with the above-mentioned indirect method. You would instead have to use the direct method.

On the other hand, you don't need to make these kind of error: Just use the tool `GenSec`, and `JGnuCashLib` will ensure that only the pre-defined values are used.

- Specifying the security by its business logic ID (the field "X-Code" in GnuCash lingo), i.e. its ISIN, CUSIP, SEDOL or similar official security identifiers (without name-space prefix).

  This approach obviously only works when you actually have filled the field "X-Code", which is typically redundant to the security's pseudo-technical ID. That bears potential for errors, so this approach will not always work. However, once you have your data in order, it is the most convenient method.

  Example:

  ```
  $ gcsh_get_sec_info.sh \
      -f test.gnucash \
      -ssm ISIN \
      -is DE000BASF111
  ```

  Notice the difference to the previous example: We also use the parameter `-is` here, but this time, it is used to search over the field "X-Code" (because the parameter `-ssm` is set to "ISIN"). Also notice that – for the time being – the exact same command is used when you have CUSIPs, SEDOLs, WKNs or something else in your X-Code. Still the above notation with "ISIN" etc. is used.[4]

::TODO Select by name (not recommended but possible for get_sec_info, and not supported for upd_sec).

[3] To get a list of all these values, just start the program with the help flag: `gcsh_get_sec_info.sh -h`.

[4] The current maintainer uses only ISINs in his own business' file, because the ISIN is the only truly global and stable identifier.